

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:
«Основы системного программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3
«Процессы, потоки и средства межпроцессного взаимодействия»

Выполнил:

Пахомов Владимир Юрьевич, студент группы N3250



(подпись)

Проверил:

Грозов Владимир Андреевич

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ЗАДАНИЕ. ВАРИАНТ 5.....	3
ХОД РАБОТЫ.....	5

ЗАДАНИЕ. ВАРИАНТ 5

Таблица 1: Переменные среды и опции командной строки, поддерживаемые программой-сервером и программой-клиентом

Опция	Переменная среды	Назначение	Значение по умолчанию	Поддерживается
-w N	lab3WAIT	Имитация работы путем приостановки обслуживающего запрос процесса/потока на N секунд. Если опция не задана, обслуживать запрос без задержки	0	Сервером
-d		Работа в режиме демона.		Сервером
-l /path/to/log	lab3LOGFILE	Путь к лог-файлу	/tmp/lab3.log	Сервером
-a ip	lab3ADDR	Адрес, на котором слушает сервер и к которому подключается клиент.		Сервером и клиентом
-p port	lab3PORT	Порт, на котором слушает сервер и к которому подключается клиент		Сервером и клиентом
-v		Вывод версии программы		Сервером и клиентом
-h		Вывод справки по опциям		Сервером и клиентом
	lab3DEBUG	Включение вывода отладочной информации		Сервером и клиентом

Таблица 2: Тип многозадачности программы-сервера

Вариант (условие)	Тип многозадачности
Количество букв в фамилии студента, выполняющего работу, нечетное .	Многопроцессорность (создание рабочих задач с помощью вызова fork)

Таблица 3: Транспортный протокол, поддерживаемый программой-сервером и программой-клиентом

Вариант (условие)	Транспортный протокол
Количество букв в имени студента, выполняющего работу, четное	TCP

Таблица 4: Прикладной протокол, поддерживаемый программой-сервером и программой-клиентом

Вариант (номер)	Описание прикладного протокола
5	<i>Запрос:</i> строка, содержащая список целых чисел в десятичной или шестнадцатеричной системах счисления, разделенных пробелами или символами табуляции (возможно, несколькими подряд), завершающаяся символом LF. <i>Ответ, если ошибок не было:</i> строка, содержащая отсортированный список чисел (разделитель — пробел, критерий сортировки — по убыванию количества разрядов в десятичном представлении числа), завершающаяся символом LF. <i>Ответ, если были ошибки:</i> строка «ERROR N», завершающаяся символом LF, где N — код ошибки.

ХОД РАБОТЫ

Исходный код Makefile:

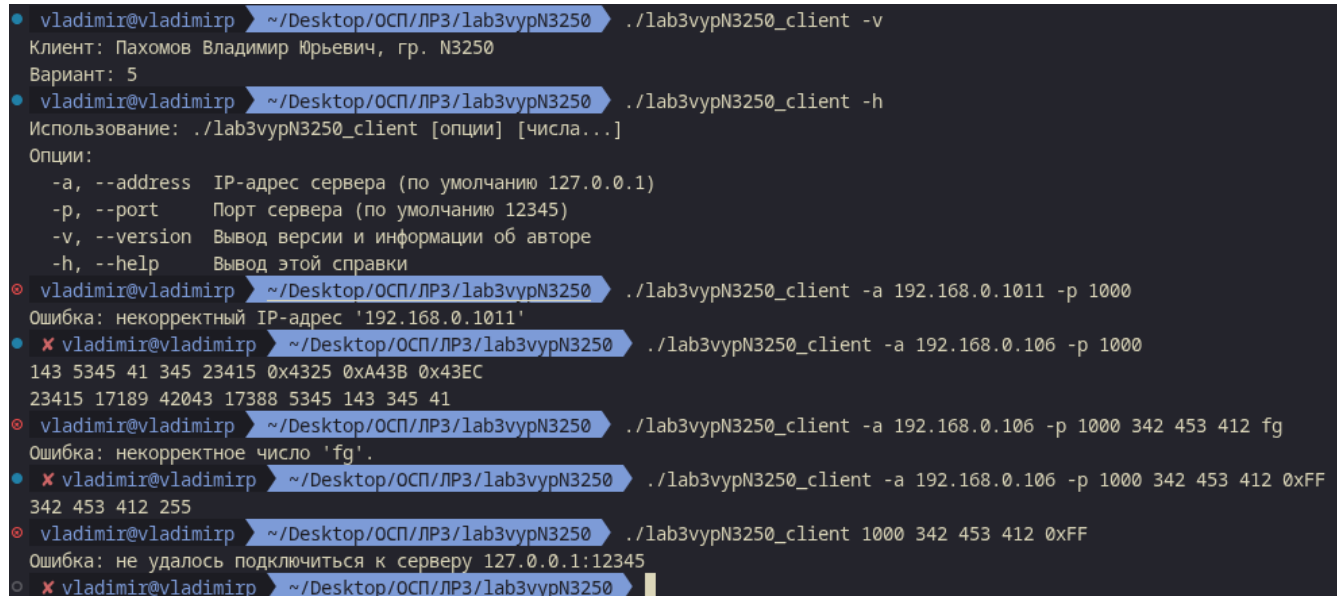
```
1 | CC = gcc
2 | CFLAGS = -Wall -Wextra -Werror -O3
3 |
4 | SERVER_TARGET = lab3vypN3250_server
5 | CLIENT_TARGET = lab3vypN3250_client
6 |
7 | SERVER_SRC = lab3vypN3250_server.c
8 | CLIENT_SRC = lab3vypN3250_client.c
9 |
10 | all: $(SERVER_TARGET) $(CLIENT_TARGET)
11 |
12 | $(SERVER_TARGET): $(SERVER_SRC)
13 |     $(CC) $(CFLAGS) $(SERVER_SRC) -o $(SERVER_TARGET)
14 |
15 | $(CLIENT_TARGET): $(CLIENT_SRC)
16 |     $(CC) $(CFLAGS) $(CLIENT_SRC) -o $(CLIENT_TARGET)
17 |
18 | clean:
19 |     rm -f $(SERVER_TARGET) $(CLIENT_TARGET) *.o
20 |
21 | test: all
22 |     @echo "=== ЗАПУСК ТЕСТОВ ==="
23 |     @echo "1. Запуск сервера в фоновом режиме на порту 23456..."
24 |     ./$(SERVER_TARGET) -p 23456 -l /tmp/lab3_test.log & \
25 |     SERVER_PID=$$! ; \
26 |     sleep 1 ; \
27 |     \
28 |     echo "2. Отправка тестового запроса (100 0x1F 5 1000 0xF)..." ; \
29 |     RESULT=$$(./$(CLIENT_TARGET) -p 23456 100 0x1F 5 1000 0xF) ; \
30 |     echo "Полученный ответ: $$RESULT" ; \
31 |     \
32 |     echo "3. Отправка некорректного запроса для проверки обработки ошибок..." ; \
33 |     ./$(CLIENT_TARGET) -p 23456 10 20abc ; \
34 |     \
35 |     echo "4. Запрос накопленной статистики у сервера (SIGUSR1)..." ; \
36 |     kill -USR1 $$SERVER_PID ; \
37 |     sleep 1 ; \
38 |     echo "Статистика в лог-файле /tmp/lab3_test.log:" ; \
39 |     tail -n 5 /tmp/lab3_test.log ; \
40 |     \
41 |     echo "5. Завершение работы тестового сервера (SIGTERM)..." ; \
42 |     kill -TERM $$SERVER_PID ; \
43 |     wait $$SERVER_PID 2>/dev/null || true ; \
44 |     echo "Тестирование завершено."
45 |
46 | .PHONY: all clean test
```

Отчет valgrind_server со строки HEAP SUMMARY:

```
==34561== HEAP SUMMARY:
==34561==      in use at exit: 0 bytes in 0 blocks
==34561==    total heap usage: 28 allocs, 28 frees, 24,488 bytes allocated
==34561==
==34561== All heap blocks were freed -- no leaks are possible
==34561==
==34561== For lists of detected and suppressed errors, rerun with: -s
==34561== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
==33838==
==33838== HEAP SUMMARY:
==33838==      in use at exit: 0 bytes in 0 blocks
==33838==    total heap usage: 27 allocs, 27 frees, 28,547 bytes allocated
==33838==
==33838== All heap blocks were freed -- no leaks are possible
==33838==
==33838== For lists of detected and suppressed errors, rerun with: -s
==33838== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Отчет valgrind_client со строки HEAP SUMMARY:

```
==34556== HEAP SUMMARY:
==34556==      in use at exit: 0 bytes in 0 blocks
==34556==    total heap usage: 3 allocs, 3 frees, 1,280 bytes allocated
==34556==
==34556== All heap blocks were freed -- no leaks are possible
==34556==
==34556== For lists of detected and suppressed errors, rerun with: -s
==34556== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```



```
• vladimir@vladimirp ~/Desktop/ОСП/ЛРЗ/lab3vypN3250 ./lab3vypN3250_client -v
Клиент: Пахомов Владимир Юрьевич, гр. N3250
Вариант: 5
• vladimir@vladimirp ~/Desktop/ОСП/ЛРЗ/lab3vypN3250 ./lab3vypN3250_client -h
Использование: ./lab3vypN3250_client [опции] [числа...]
Опции:
  -a, --address IP-адрес сервера (по умолчанию 127.0.0.1)
  -p, --port    Порт сервера (по умолчанию 12345)
  -v, --version Вывод версии и информации об авторе
  -h, --help    Вывод этой справки
• vladimir@vladimirp ~/Desktop/ОСП/ЛРЗ/lab3vypN3250 ./lab3vypN3250_client -a 192.168.0.1011 -p 1000
Ошибка: некорректный IP-адрес '192.168.0.1011'
• ✗ vladimir@vladimirp ~/Desktop/ОСП/ЛРЗ/lab3vypN3250 ./lab3vypN3250_client -a 192.168.0.106 -p 1000
143 5345 41 345 23415 0x4325 0xA43B 0x43EC
23415 17189 42043 17388 5345 143 345 41
• vladimir@vladimirp ~/Desktop/ОСП/ЛРЗ/lab3vypN3250 ./lab3vypN3250_client -a 192.168.0.106 -p 1000 342 453 412 fg
Ошибка: некорректное число 'fg'.
• ✗ vladimir@vladimirp ~/Desktop/ОСП/ЛРЗ/lab3vypN3250 ./lab3vypN3250_client -a 192.168.0.106 -p 1000 342 453 412 0xFF
342 453 412 255
• vladimir@vladimirp ~/Desktop/ОСП/ЛРЗ/lab3vypN3250 ./lab3vypN3250_client 1000 342 453 412 0xFF
Ошибка: не удалось подключиться к серверу 127.0.0.1:12345
• ✗ vladimir@vladimirp ~/Desktop/ОСП/ЛРЗ/lab3vypN3250
```

Рисунок 1: Пример работы клиента

```

• vladimir@vladimirp ~/Desktop/ОСП/ЛР3/lab3vypN3250 ./lab3vypN3250_server -v
Сервер: Пахомов Владимир Юрьевич, гр. N3250
Вариант: 5 (Многопроцессный TCP)
• vladimir@vladimirp ~/Desktop/ОСП/ЛР3/lab3vypN3250 ./lab3vypN3250_server -h
Использование: ./lab3vypN3250_server [опции]
Опции:
  -a, --address IP-адрес для прослушивания (по умолчанию: 127.0.0.1)
  -p, --port    Порт для прослушивания (по умолчанию: 12345)
  -l, --log     Путь к лог-файлу (по умолчанию: /tmp/lab3.log)
  -w, --wait    Имитация работы (задержка N секунд)
  -d, --daemon  Запуск в режиме демона
  -v, --version Вывод версии и информации об авторе
  -h, --help    Вывод этой справки
• vladimir@vladimirp ~/Desktop/ОСП/ЛР3/lab3vypN3250 ./lab3vypN3250_server -a 192.168.0.1000
Ошибка: некорректный IP-адрес '192.168.0.1000'
• ✗ vladimir@vladimirp ~/Desktop/ОСП/ЛР3/lab3vypN3250 ./lab3vypN3250_server -a 192.168.0.104
-p 1000
Ошибка при привязке сокета (bind): Cannot assign requested address
• ✗ vladimir@vladimirp ~/Desktop/ОСП/ЛР3/lab3vypN3250 ./lab3vypN3250_server -a 192.168.0.106 -p 1000
Ошибка при привязке сокета (bind): Permission denied
• ✗ vladimir@vladimirp ~/Desktop/ОСП/ЛР3/lab3vypN3250 sudo ./lab3vypN3250_server -a 192.168.0.106 -p 1000
[sudo] password for vladimir:

^C--- СТАТИСТИКА РАБОТЫ СЕРВЕРА ---
Время работы: 486 сек
Успешно обслужено запросов: 4
Запросов с ошибками: 0
-----
• vladimir@vladimirp ~/Desktop/ОСП/ЛР3/lab3vypN3250

```

Рисунок 2: Пример работы сервера

```

vladimir@vladimirp ~/Desktop/ОСП/ЛР3/lab3vypN3250 make test
=== ЗАПУСК ТЕСТОВ ===
1. Запуск сервера в фоновом режиме на порту 23456...
./lab3vypN3250_server -p 23456 -l /tmp/lab3_test.log & \
SERVER_PID=$! ; \
sleep 1 ; \
\
echo "2. Отправка тестового запроса (100 0x1F 5 1000 0xF)..." ; \
RESULT=$(./lab3vypN3250_client -p 23456 100 0x1F 5 1000 0xF) ; \
echo "Полученный ответ: $RESULT" ; \
\
echo "3. Отправка некорректного запроса для проверки обработки ошибок..." ; \
./lab3vypN3250_client -p 23456 10 20abc ; \
\
echo "4. Запрос накопленной статистики у сервера (SIGUSR1)..." ; \
kill -USR1 $SERVER_PID ; \
sleep 1 ; \
echo "Статистика в лог-файле /tmp/lab3_test.log:" ; \
tail -n 5 /tmp/lab3_test.log ; \
\
echo "5. Завершение работы тестового сервера (SIGTERM)..." ; \
kill -TERM $SERVER_PID ; \
wait $SERVER_PID 2>/dev/null || true ; \
echo "Тестирование завершено."
2. Отправка тестового запроса (100 0x1F 5 1000 0xF)...
Полученный ответ: 1000 100 31 15 5
3. Отправка некорректного запроса для проверки обработки ошибок...
Ошибка: некорректное число '20abc'.
4. Запрос накопленной статистики у сервера (SIGUSR1)...
--- СТАТИСТИКА РАБОТЫ СЕРВЕРА ---
Время работы: 1 сек
Успешно обслужено запросов: 1
Запросов с ошибками: 0
-----
Статистика в лог-файле /tmp/lab3_test.log:
[24.05.26 22:41:23] --- СТАТИСТИКА РАБОТЫ СЕРВЕРА ---
Время работы: 1 сек
Успешно обслужено запросов: 1
Запросов с ошибками: 0
-----
5. Завершение работы тестового сервера (SIGTERM)...
--- СТАТИСТИКА РАБОТЫ СЕРВЕРА ---
Время работы: 2 сек
Успешно обслужено запросов: 1
Запросов с ошибками: 0
-----
Тестирование завершено.
vladimir@vladimirp ~/Desktop/ОСП/ЛР3/lab3vypN3250

```

Рисунок 3: Результат работы тестов из Makefile

Исходный код программы lab3vypN3250_client:

```
1  | #include <getopt.h>
2  | #include <stdio.h>
3  | #include <stdlib.h>
4  | #include <string.h>
5  | #include <unistd.h>
6  | #include <sys/socket.h>
7  | #include <netinet/in.h>
8  | #include <arpa/inet.h>
9  |
10 |
11 | int main(int argc, char *argv[]) {
12 |     int debug = (getenv("lab3DEBUG") != NULL);
13 |
14 |     char *address = "127.0.0.1";
15 |     int port = 12345;
16 |
17 |     char *env_address = getenv("lab3ADDR");
18 |     char *env_port = getenv("lab3PORT");
19 |     char *endptr;
20 |
21 |
22 |     if (env_address != NULL) address = env_address;
23 |     if (env_port != NULL) {
24 |         long val = strtol(env_port, &endptr, 10);
25 |         if (*endptr == '\0' && val > 0 && val <= 65535) {
26 |             port = (int)val;
27 |         } else {
28 |             fprintf(stderr, "Ошибка: неверный формат порта: %s\n", env_port);
29 |             return 1;
30 |         }
31 |     }
32 |
33 |     static struct option longopts[] = {
34 |         {"address", required_argument, NULL, 'a'},
35 |         {"port", required_argument, NULL, 'p'},
36 |         {"verbose", no_argument, NULL, 'v'},
37 |         {"help", no_argument, NULL, 'h'},
38 |         {NULL, 0, NULL, 0}
39 |     };
40 |
41 |
42 |     char *author = "Клиент: Пахомов Владимир Юрьевич, гр. N3250\nВариант: 5\n";
43 |     char *help = "Использование: ./lab3vypN3250_client [опции] [числа...]\n"
44 |                 "Опции:\n"
45 |                 "  -a, --address  IP-адрес сервера (по умолчанию 127.0.0.1)\n"
46 |                 "  -p, --port     Порт сервера (по умолчанию 12345)\n"
47 |                 "  -v, --version  Вывод версии и информации об авторе\n"
48 |                 "  -h, --help     Вывод этой справки\n";
49 |
50 |     int opt;
51 |     while ((opt = getopt_long(argc, argv, "a:p:vh", longopts, NULL)) != -1) {
52 |         switch (opt) {
53 |             case 'a':
54 |                 address = optarg;
55 |                 break;
56 |             case 'p': {
57 |                 char *endptr;
58 |                 long val = strtol(optarg, &endptr, 10);
59 |                 if (*endptr != '\0' || val <= 0 || val > 65535) {
60 |                     fprintf(stderr, "Ошибка: некорректное значение порта '%s'. Должно быть
число от 1 до 65535.\n", optarg);
61 |                     return 1;
62 |                 }
63 |                 port = (int)val;
64 |                 break;
65 |             }
66 |             case 'v':
67 |                 printf("%s", author);
68 |                 return 0;
69 |             case 'h':
70 |                 printf("%s", help);
```

```

71         return 0;
72     default:
73         fprintf(stderr, "%s", help);
74         return 1;
75     }
76 }
77
78 if (debug) {
79     fprintf(stderr, "[DEBUG] Подключение к %s:%d\n", address, port);
80 }
81
82 struct in_addr test_addr;
83 if (inet_pton(AF_INET, address, &test_addr) <= 0) {
84     fprintf(stderr, "Ошибка: некорректный IP-адрес '%s'\n", address);
85     return 1;
86 }
87
88 size_t query_cap = 128;
89 char *query = malloc(query_cap);
90 if (query == NULL) {
91     fprintf(stderr, "Ошибка: не удалось выделить память для запроса\n");
92     return 1;
93 }
94 query[0] = '\0';
95 size_t query_len = 0;
96
97 if (optind < argc) {
98     for (int i = optind; i < argc; i++) {
99         char *endptr;
100         strtoul(argv[i], &endptr, 0);
101
102         if (*endptr != '\0' || endptr == argv[i]) {
103             fprintf(stderr, "Ошибка: некорректное число '%s'.\n", argv[i]);
104             free(query);
105             return 1;
106         }
107
108         if (debug) {
109             fprintf(stderr, "[DEBUG] Число: %s\n", argv[i]);
110         }
111
112         size_t arg_len = strlen(argv[i]);
113         while (query_len + arg_len + 2 > query_cap) {
114             query_cap *= 2;
115             char *temp = realloc(query, query_cap);
116             if (temp == NULL) {
117                 fprintf(stderr, "Ошибка: не удалось выделить память для запроса\n");
118                 free(query);
119                 return 1;
120             }
121             query = temp;
122         }
123
124         strcpy(query + query_len, argv[i]);
125         query_len += arg_len;
126
127         if (i < argc - 1) {
128             query[query_len++] = ' ';
129         } else {
130             query[query_len++] = '\n';
131         }
132         query[query_len] = '\0';
133     }
134 } else {
135     if (debug) {
136         fprintf(stderr, "[DEBUG] Аргументы не переданы. Ожидание ввода из stdin...\n");
137     }
138
139     size_t cap = 128;
140     size_t buf_len = 0;
141     char *buf = malloc(cap);
142     if (buf == NULL) {
143         fprintf(stderr, "Ошибка: не удалось выделить память для ввода\n");
144         free(query);

```

```

145         return 1;
146     }
147
148     int c;
149     while ((c = fgetc(stdin)) != EOF && c != '\n') {
150         if (buf_len + 2 >= cap) {
151             cap *= 2;
152             char *temp = realloc(buf, cap);
153             if (temp == NULL) {
154                 fprintf(stderr, "Ошибка: не удалось выделить память для ввода\n");
155                 free(buf);
156                 free(query);
157                 return 1;
158             }
159             buf = temp;
160         }
161         buf[buf_len++] = (char)c;
162     }
163
164     buf[buf_len++] = '\n';
165     buf[buf_len] = '\0';
166
167     free(query);
168     query = buf;
169     query_len = buf_len;
170
171     char *query_copy = malloc(query_len + 1);
172     if (query_copy == NULL) {
173         fprintf(stderr, "Ошибка: не удалось выделить память для валидации\n");
174         free(query);
175         return 1;
176     }
177
178     strcpy(query_copy, query);
179
180     char *token = strtok(query_copy, " \\t\\v\\f\\r\\n");
181     if (token == NULL) {
182         fprintf(stderr, "Ошибка: пустой запрос\n");
183         free(query);
184         free(query_copy);
185         return 1;
186     }
187
188     while (token != NULL) {
189         char *endptr;
190         strtoul(token, &endptr, 0);
191         if (*endptr != '\0' || endptr == token) {
192             fprintf(stderr, "Ошибка: некорректное число '%s'\n", token);
193             free(query);
194             free(query_copy);
195             return 1;
196         }
197         token = strtok(NULL, " \\t\\v\\f\\r\\n");
198     }
199     free(query_copy);
200
201 }
202
203 if (debug) {
204     fprintf(stderr, "[DEBUG] Сформированный запрос: %s", query);
205 }
206
207 int sock = socket(AF_INET, SOCK_STREAM, 0);
208 if (sock < 0) {
209     fprintf(stderr, "Ошибка: не удалось создать сокет\n");
210     free(query);
211     return 1;
212 }
213
214 struct sockaddr_in server_addr;
215 memset(&server_addr, 0, sizeof(server_addr));
216 server_addr.sin_family = AF_INET;
217 server_addr.sin_port = htons(port);
218

```

```

219     if (inet_pton(AF_INET, address, &server_addr.sin_addr) <= 0) {
220         fprintf(stderr, "Ошибка: некорректный IP-адрес '%s'\n", address);
221         free(query);
222         close(sock);
223         return 1;
224     }
225
226     if (debug) {
227         fprintf(stderr, "[DEBUG] Установка TCP соединения с %s:%d...\n", address, port);
228     }
229
230     if (connect(sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
231         fprintf(stderr, "Ошибка: не удалось подключиться к серверу %s:%d\n", address, port);
232         free(query);
233         close(sock);
234         return 1;
235     }
236
237     if (debug) {
238         fprintf(stderr, "[DEBUG] Соединение установлено. Отправка запроса...\n");
239     }
240
241     ssize_t sent = write(sock, query, query_len);
242     if (sent < 0) {
243         fprintf(stderr, "Ошибка: не удалось отправить запрос\n");
244         free(query);
245         close(sock);
246         return 1;
247     }
248
249     if (debug) {
250         fprintf(stderr, "[DEBUG] Запрос отправлен. Ожидание ответа...\n");
251     }
252     if (shutdown(sock, SHUT_WR) < 0) {
253         fprintf(stderr, "Ошибка: не удалось закрыть запись в сокете\n");
254         free(query);
255         close(sock);
256         return 1;
257     }
258
259     size_t resp_cap = 128;
260     size_t resp_len = 0;
261     char *response = malloc(resp_cap);
262     if (response == NULL) {
263         fprintf(stderr, "Ошибка: не удалось выделить память для ответа\n");
264         free(query);
265         close(sock);
266         return 1;
267     }
268
269     while (1) {
270         if (resp_len + 64 >= resp_cap) {
271             resp_cap *= 2;
272             char *temp = realloc(response, resp_cap);
273             if (temp == NULL) {
274                 fprintf(stderr, "Ошибка: не удалось выделить память для ответа\n");
275                 free(response);
276                 free(query);
277                 close(sock);
278                 return 1;
279             }
280             response = temp;
281         }
282
283         ssize_t received = read(sock, response + resp_len, 64);
284         if (received < 0) {
285             fprintf(stderr, "Ошибка: не удалось прочитать ответ\n");
286             free(response);
287             free(query);
288             close(sock);
289             return 1;
290         }
291         if (received == 0) {
292             break;

```

```
293 |     }
294 |     resp_len += received;
295 | }
296 | response[resp_len] = '\0';
297 |
298 | if (debug) {
299 |     fprintf(stderr, "[DEBUG] Ответ получен (%zu байт). Закрытие соединения...\n",
resp_len);
300 | }
301 |
302 | printf("%s", response);
303 | free(response);
304 | close(sock);
305 | free(query);
306 |
307 | return 0;
308 |
309 | }
```

Исходный код программы lab3vypN3250_server.c:

```
1  | #include <sys/stat.h>
2  | #define _GNU_SOURCE
3  | #include <stdio.h>
4  | #include <stdlib.h>
5  | #include <string.h>
6  | #include <unistd.h>
7  | #include <getopt.h>
8  | #include <signal.h>
9  | #include <errno.h>
10 | #include <time.h>
11 | #include <fcntl.h>
12 | #include <sys/socket.h>
13 | #include <sys/types.h>
14 | #include <sys/wait.h>
15 | #include <sys/mman.h>
16 | #include <netinet/in.h>
17 | #include <arpa/inet.h>
18 |
19 | typedef struct {
20 |     time_t start_time;
21 |     unsigned long success_count;
22 |     unsigned long error_count;
23 | } ServerStats;
24 |
25 | volatile sig_atomic_t keep_running = 1;
26 | volatile sig_atomic_t dump_stats = 0;
27 |
28 | ServerStats *stats = NULL;
29 | char *log_path = "/tmp/lab3.log";
30 | int debug = 0;
31 |
32 | void write_log(const char *message, int is_debug_msg) {
33 |     if (is_debug_msg && !debug) {
34 |         return;
35 |     }
36 |
37 |     FILE *f = fopen(log_path, "a");
38 |     if (!f) {
39 |         return;
40 |     }
41 |
42 |     time_t now = time(NULL);
43 |     struct tm *tm_info = localtime(&now);
44 |     char time_buf[26];
45 |     strftime(time_buf, sizeof(time_buf), "%d.%m.%y %H:%M:%S", tm_info);
46 |
47 |     fprintf(f, "[%s] %s\n", time_buf, message);
48 |     fclose(f);
49 | }
50 |
51 | void print_statistics(void) {
52 |     if (stats == NULL) return;
53 |
54 |     time_t now = time(NULL);
55 |     double elapsed = difftime(now, stats->start_time);
56 |
57 |     char stat_buf[256];
58 |     snprintf(stat_buf, sizeof(stat_buf),
59 |              "---- СТАТИСТИКА РАБОТЫ СЕРВЕРА ----\n"
60 |              "Время работы: %.0f сек\n"
61 |              "Успешно обслужено запросов: %lu\n"
62 |              "Запросов с ошибками: %lu\n"
63 |              "-----",
64 |              elapsed, stats->success_count, stats->error_count);
65 |
66 |     write_log(stat_buf, 0);
67 |
68 |     fprintf(stderr, "%s\n", stat_buf);
69 | }
70 |
71 | void handle_signal(int sig) {
```

```

72     if (sig == SIGINT || sig == SIGTERM || sig == SIGQUIT) {
73         keep_running = 0;
74     } else if (sig == SIGUSR1) {
75         dump_stats = 1;
76     }
77 }
78
79 void handle_sigchld(int sig) {
80     (void)sig;
81     int saved_errno = errno;
82     while (waitpid(-1, NULL, WNOHANG) > 0);
83     errno = saved_errno;
84 }
85
86 void daemonize(void) {
87     pid_t pid = fork();
88     if (pid < 0) {
89         perror("fork");
90         exit(EXIT_FAILURE);
91     }
92     if (pid > 0) {
93         exit(EXIT_SUCCESS);
94     }
95
96     if (setsid() < 0) {
97         perror("setsid");
98         exit(EXIT_FAILURE);
99     }
100
101     pid = fork();
102     if (pid < 0) exit(EXIT_FAILURE);
103     if (pid > 0) exit(EXIT_SUCCESS);
104
105     umask(0);
106     if (chdir("/") < 0) {
107         perror("chdir");
108         exit(EXIT_FAILURE);
109     }
110
111     close(STDIN_FILENO);
112     close(STDOUT_FILENO);
113     close(STDERR_FILENO);
114
115     open("/dev/null", O_RDONLY);
116     open("/dev/null", O_WRONLY);
117     open("/dev/null", O_RDWR);
118 }
119
120 int count_decimal_digits(long long val) {
121     if (val == 0) return 1;
122     unsigned long long abs_val = (val < 0) ? -val : val;
123     int count = 0;
124     while (abs_val > 0) {
125         count++;
126         abs_val /= 10;
127     }
128     return count;
129 }
130
131 typedef struct {
132     long long value;
133     int digits;
134     int original_index;
135 } NumberItem;
136
137 int compare_items(const void *a, const void *b) {
138     const NumberItem *ia = (const NumberItem *)a;
139     const NumberItem *ib = (const NumberItem *)b;
140     if (ia->digits != ib->digits) {
141         return ib->digits - ia->digits;
142     }
143     return ia->original_index - ib->original_index;
144 }
145

```

```

146 | char *process_request(const char *request, size_t *out_len) {
147 |     char *req_copy = strdup(request);
148 |     if (!req_copy) {
149 |         *out_len = snprintf(NULL, 0, "ERROR 2\n");
150 |         char *err_res = malloc(*out_len + 1);
151 |         if (err_res) sprintf(err_res, "ERROR 2\n");
152 |         return err_res;
153 |     }
154 |
155 |     size_t items_cap = 16;
156 |     size_t items_count = 0;
157 |     NumberItem *items = malloc(items_cap * sizeof(NumberItem));
158 |     if (!items) {
159 |         free(req_copy);
160 |         *out_len = snprintf(NULL, 0, "ERROR 2\n");
161 |         char *err_res = malloc(*out_len + 1);
162 |         if (err_res) sprintf(err_res, "ERROR 2\n");
163 |         return err_res;
164 |     }
165 |
166 |     char *saveptr;
167 |     char *token = strtok_r(req_copy, " \\t\\v\\f\\r\\n", &saveptr);
168 |     int has_invalid_tokens = 0;
169 |
170 |     while (token != NULL) {
171 |         char *endptr;
172 |         long long val = strtoll(token, &endptr, 0);
173 |
174 |         if (*endptr != '\\0' || endptr == token) {
175 |             has_invalid_tokens = 1;
176 |             break;
177 |         }
178 |
179 |         if (items_count >= items_cap) {
180 |             items_cap *= 2;
181 |             NumberItem *temp = realloc(items, items_cap * sizeof(NumberItem));
182 |             if (!temp) {
183 |                 has_invalid_tokens = 2;
184 |                 break;
185 |             }
186 |             items = temp;
187 |         }
188 |
189 |         items[items_count].value = val;
190 |         items[items_count].digits = count_decimal_digits(val);
191 |         items[items_count].original_index = (int)items_count;
192 |         items_count++;
193 |
194 |         token = strtok_r(NULL, " \\t\\v\\f\\r\\n", &saveptr);
195 |     }
196 |
197 |     free(req_copy);
198 |
199 |     char *res_str = NULL;
200 |     if (has_invalid_tokens == 1 || items_count == 0) {
201 |         stats->error_count++;
202 |         *out_len = asprintf(&res_str, "ERROR 1\n");
203 |     } else if (has_invalid_tokens == 2) {
204 |         stats->error_count++;
205 |         *out_len = asprintf(&res_str, "ERROR 2\n");
206 |     } else {
207 |         qsort(items, items_count, sizeof(NumberItem), compare_items);
208 |
209 |         size_t res_cap = 128;
210 |         res_str = malloc(res_cap);
211 |         if (!res_str) {
212 |             stats->error_count++;
213 |             *out_len = asprintf(&res_str, "ERROR 2\n");
214 |         } else {
215 |             size_t res_len = 0;
216 |             res_str[0] = '\\0';
217 |
218 |             for (size_t i = 0; i < items_count; i++) {
219 |                 char num_buf[32];

```

```

220         int printed = snprintf(num_buf, sizeof(num_buf), "%lld", items[i].value);
221
222         while (res_len + printed + 2 > res_cap) {
223             res_cap *= 2;
224             char *temp = realloc(res_str, res_cap);
225             if (!temp) {
226                 free(res_str);
227                 stats->error_count++;
228                 *out_len = asprintf(&res_str, "ERROR 2\n");
229                 free(items);
230                 return res_str;
231             }
232             res_str = temp;
233         }
234
235         strcpy(res_str + res_len, num_buf);
236         res_len += printed;
237
238         if (i < items_count - 1) {
239             res_str[res_len++] = ' ';
240         } else {
241             res_str[res_len++] = '\n';
242         }
243         res_str[res_len] = '\0';
244     }
245     stats->success_count++;
246     *out_len = res_len;
247 }
248 }
249
250 free(items);
251 return res_str;
252 }
253
254 int main(int argc, char *argv[]) {
255     debug = (getenv("lab3DEBUG") != NULL);
256
257     int wait_time = 0;
258     int is_daemon = 0;
259     char *address = "127.0.0.1";
260     int port = 12345;
261
262     char *env_address = getenv("lab3ADDR");
263     char *env_port = getenv("lab3PORT");
264     char *env_log = getenv("lab3LOGFILE");
265     char *env_wait = getenv("lab3WAIT");
266
267     if (env_address != NULL) address = env_address;
268     if (env_log != NULL) log_path = env_log;
269
270     if (env_port != NULL) {
271         char *endptr;
272         long val = strtol(env_port, &endptr, 10);
273         if (*endptr == '\0' && val > 0 && val <= 65535) port = (int)val;
274     }
275     if (env_wait != NULL) {
276         char *endptr;
277         long val = strtol(env_wait, &endptr, 10);
278         if (*endptr == '\0' && val >= 0) wait_time = (int)val;
279     }
280
281     stats = mmap(NULL, sizeof(ServerStats), PROT_READ | PROT_WRITE,
282                 MAP_SHARED | MAP_ANONYMOUS, -1, 0);
283     if (stats == MAP_FAILED) {
284         perror("mmap");
285         return 1;
286     }
287     stats->start_time = time(NULL);
288     stats->success_count = 0;
289     stats->error_count = 0;
290
291     static struct option longopts[] = {
292         {"wait", required_argument, NULL, 'w'},
293         {"daemon", no_argument, NULL, 'd'},

```

```

294     {"log", required_argument, NULL, 'l'},
295     {"address", required_argument, NULL, 'a'},
296     {"port", required_argument, NULL, 'p'},
297     {"version", no_argument, NULL, 'v'},
298     {"help", no_argument, NULL, 'h'},
299     {NULL, 0, NULL, 0}
300 };
301
302 char *author = "Сервер: Пахомов Владимир Юрьевич, гр. N3250\nВариант: 5\n";
303 char *help = "Использование: ./lab3vypN3250_server [опции]\n"
304             "Опции:\n"
305             "  -a, --address  IP-адрес для прослушивания (по умолчанию: 127.0.0.1)\n"
306             "  -p, --port     Порт для прослушивания (по умолчанию: 12345)\n"
307             "  -l, --log      Путь к лог-файлу (по умолчанию: /tmp/lab3.log)\n"
308             "  -w, --wait     Имитация работы (задержка N секунд)\n"
309             "  -d, --daemon   Запуск в режиме демона\n"
310             "  -v, --version  Вывод версии и информации об авторе\n"
311             "  -h, --help     Вывод этой справки\n";
312
313 int opt;
314 while ((opt = getopt_long(argc, argv, "w:dl:a:p:vh", longopts, NULL)) != -1) {
315     switch (opt) {
316         case 'a':
317             address = optarg;
318             break;
319         case 'p': {
320             char *endptr;
321             long val = strtol(optarg, &endptr, 10);
322             if (*endptr == '\0' && val > 0 && val <= 65535) port = (int)val;
323             break;
324         }
325         case 'l':
326             log_path = optarg;
327             break;
328         case 'w': {
329             char *endptr;
330             long val = strtol(optarg, &endptr, 10);
331             if (*endptr == '\0' && val >= 0) wait_time = (int)val;
332             break;
333         }
334         case 'd':
335             is_daemon = 1;
336             break;
337         case 'v':
338             printf("%s", author);
339             munmap(stats, sizeof(ServerStats));
340             return 0;
341         case 'h':
342             printf("%s", help);
343             munmap(stats, sizeof(ServerStats));
344             return 0;
345         default:
346             fprintf(stderr, "%s", help);
347             munmap(stats, sizeof(ServerStats));
348             return 1;
349     }
350 }
351
352
353 struct in_addr test_addr;
354 if (inet_pton(AF_INET, address, &test_addr) <= 0) {
355     fprintf(stderr, "Ошибка: некорректный IP-адрес '%s'\n", address);
356     return 1;
357 }
358
359 if (is_daemon) {
360     daemonize();
361 }
362
363 write_log("--- СЕРВЕР ЗАПУЩЕН ---", 0);
364 char start_info[256];
365 snprintf(start_info, sizeof(start_info), "Слушаем на %s:%d, Имитация ожидания: %d сек",
address, port, wait_time);
366 write_log(start_info, 0);

```

```

367 | struct sigaction sa;
368 | memset(&sa, 0, sizeof(sa));
369 | sa.sa_handler = handle_signal;
370 | sigemptyset(&sa.sa_mask);
371 |
372 |
373 | sigaction(SIGINT, &sa, NULL);
374 | sigaction(SIGTERM, &sa, NULL);
375 | sigaction(SIGQUIT, &sa, NULL);
376 | sigaction(SIGUSR1, &sa, NULL);
377 |
378 | struct sigaction sa_chld;
379 | memset(&sa_chld, 0, sizeof(sa_chld));
380 | sa_chld.sa_handler = handle_sigchld;
381 | sigemptyset(&sa_chld.sa_mask);
382 | sa_chld.sa_flags = SA_RESTART | SA_NOCLDSTOP;
383 | sigaction(SIGCHLD, &sa_chld, NULL);
384 |
385 | int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
386 | if (listen_fd < 0) {
387 |     write_log("Ошибка при создании сокета", 0);
388 |     munmap(stats, sizeof(ServerStats));
389 |     return 1;
390 | }
391 |
392 | int optval = 1;
393 | setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(optval));
394 |
395 | struct sockaddr_in serv_addr;
396 | memset(&serv_addr, 0, sizeof(serv_addr));
397 | serv_addr.sin_family = AF_INET;
398 | serv_addr.sin_port = htons(port);
399 | inet_pton(AF_INET, address, &serv_addr.sin_addr);
400 |
401 | if (bind(listen_fd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
402 |     perror("Ошибка при привязке сокета (bind)");
403 |     write_log("Ошибка при привязке сокета (bind)", 0);
404 |     close(listen_fd);
405 |     munmap(stats, sizeof(ServerStats));
406 |     return 1;
407 | }
408 |
409 | if (listen(listen_fd, 10) < 0) {
410 |     perror("Ошибка при вызове listen");
411 |     write_log("Ошибка при вызове listen", 0);
412 |     close(listen_fd);
413 |     munmap(stats, sizeof(ServerStats));
414 |     return 1;
415 | }
416 |
417 | while (keep_running) {
418 |     if (dump_stats) {
419 |         print_statistics();
420 |         dump_stats = 0;
421 |     }
422 |
423 |     struct sockaddr_in client_addr;
424 |     socklen_t client_len = sizeof(client_addr);
425 |
426 |     int client_fd = accept(listen_fd, (struct sockaddr *)&client_addr, &client_len);
427 |     if (client_fd < 0) {
428 |         if (errno == EINTR) {
429 |             continue;
430 |         }
431 |         write_log("Ошибка при accept", 0);
432 |         continue;
433 |     }
434 |
435 |     char client_ip[INET_ADDRSTRLEN];
436 |     inet_ntop(AF_INET, &client_addr.sin_addr, client_ip, sizeof(client_ip));
437 |
438 |     char conn_msg[128];
439 |     snprintf(conn_msg, sizeof(conn_msg), "Новое подключение с адреса: %s", client_ip);
440 |     write_log(conn_msg, 0);

```

```

441 |
442 |     pid_t pid = fork();
443 |     if (pid < 0) {
444 |         write_log("Ошибка при вызове fork()", 0);
445 |         close(client_fd);
446 |     } else if (pid == 0) {
447 |         close(listen_fd);
448 |
449 |         if (wait_time > 0) {
450 |             char wait_msg[128];
451 |             snprintf(wait_msg, sizeof(wait_msg), "[DEBUG] Имитация работы: задержка %d
cek...", wait_time);
452 |             write_log(wait_msg, 1);
453 |             sleep(wait_time);
454 |         }
455 |
456 |         size_t req_cap = 128;
457 |         size_t req_len = 0;
458 |         char *request = malloc(req_cap);
459 |         if (!request) {
460 |             close(client_fd);
461 |             exit(EXIT_FAILURE);
462 |         }
463 |
464 |         while (1) {
465 |             if (req_len + 64 >= req_cap) {
466 |                 req_cap *= 2;
467 |                 char *temp = realloc(request, req_cap);
468 |                 if (!temp) break;
469 |                 request = temp;
470 |             }
471 |             ssize_t n = read(client_fd, request + req_len, 64);
472 |             if (n <= 0) break;
473 |             req_len += n;
474 |         }
475 |         request[req_len] = '\0';
476 |
477 |         char req_log[256];
478 |         snprintf(req_log, sizeof(req_log), "Получен запрос от %s: %s", client_ip,
request);
479 |         write_log(req_log, 1);
480 |
481 |         size_t res_len = 0;
482 |         char *response = process_request(request, &res_len);
483 |
484 |         if (response) {
485 |             write(client_fd, response, res_len);
486 |             free(response);
487 |         }
488 |
489 |         free(request);
490 |         close(client_fd);
491 |
492 |         char dis_msg[128];
493 |         snprintf(dis_msg, sizeof(dis_msg), "Завершено обслуживание запроса от %s",
client_ip);
494 |         write_log(dis_msg, 0);
495 |
496 |         exit(EXIT_SUCCESS);
497 |     } else {
498 |         close(client_fd);
499 |     }
500 | }
501 |
502 | write_log("--- СЕРВЕР ЗАВЕРШАЕТ РАБОТУ ---", 0);
503 | print_statistics();
504 |
505 | close(listen_fd);
506 | munmap(stats, sizeof(ServerStats));
507 |
508 | return 0;
509 | }

```

